CS/SE/CE 3354 Software Engineering
Team Final Project Specifications

Deliverable #2 Report

1. **Project Title:**
   Dot (To-do list application)
   Link to GitHub Repository: https://github.com/3354-Team1/3354-Team1.git
   Link to Slides: https://docs.google.com/presentation

2. **Delegation of tasks:**
   Everyone:
   - Participate in and record presentation
   - Review and finalize deliverable

   Anila Narula:
   - Write a conclusion evaluating all the work done
   - Create slides for: Challenges, lessons learned, and conclusion

   Bryan Fuentes-Cortez:
   - Make an estimation of the schedule for the project
   - Create slides for: Project timeline

   Dillon Odum:
   - Delegate tasks
   - Create slides for: Deliverable #1 content, Objective of the project, and Prototype of proposed application (like with Figma)
   - Finalize overall slideshow design
   - Submit everything required to eLearning

   Ryan Jones:
   - Put everything required and already submitted in Deliverable #1 on Github for Deliverable #2
   - Complete a test plan for the software
   - Create slides for: Demonstrate true and false test-case execution from in test plan
   - Commit everything required to GitHub

   Willie Lin:
   - Make an estimation of the cost, effort, and pricing along with estimations of hardware, software, and personnel costs
   - Create slides for: Cost estimation

   Zubiyaa Khan:
   - Make a comparison of the project with similar designs
   - Create slides for: Comparison with similar designs

3. **Everything required and already submitted in Deliverable #1:**
   a. **Delegation of tasks from Deliverable #1:**
      Everyone:
      - Create GitHub accounts
      - Create project title
      - Review and finalize deliverable

Anila Narula:
- Add TAs as collaborators to GitHub repository
- Software process model selection and justification
- Create sequence diagram
- Create architectural design

Bryan Fuentes-Cortez:
- Software process model selection and justification
- Create sequence diagram
- Create architectural design

Dillon Odum:
- Add second commit to GitHub with project scope document
- Delegate tasks
- List software requirements (functional and non-functional)
- Create use case diagram
- Create class diagram
- Create architectural design

Ryan Jones:
- Create team GitHub repository and add team members as collaborators
- Software process model selection and justification
- Create sequence diagram
- Create architectural design

Willie Lin:
- Add first commit to GitHub with README file
- Software process model selection and justification
- Create sequence diagram
- Create architectural design

Zubiyaa Khan:
- Software process model selection and justification
- List project assumptions
- Create sequence diagram
- Create architectural design

b. Assumptions:
- The system will support a single user interacting with the application.
- To-do list tasks will be stored on the user's device locally using file based storage.
- The system will manage around 100 to-do list tasks without any noticeable decrease in performance.
- Categories for to-do list tasks can contain multiple tasks, while tasks can only belong in up to one category.
- The system will automatically load all saved tasks when the application starts.

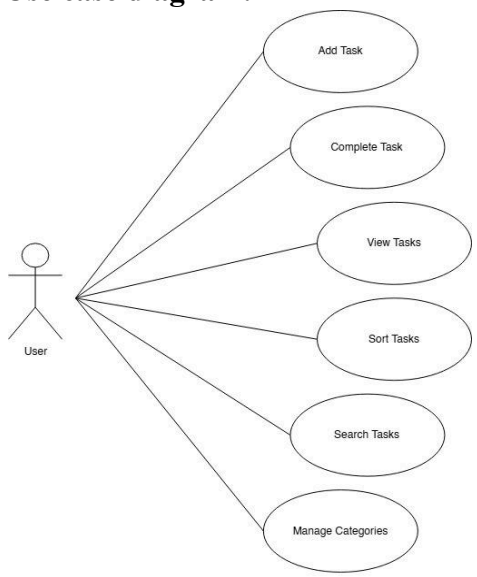
c. **Addressed feedback:**
We were told “Well Done” for our project proposal, so there really isn’t any feedback to address.

d. **Software process model used:**
The software process model that was employed for our project was the incremental development model. The incremental development model allows the system to be developed in small functional components called increments, with each increment adding features to the system until the final application is complete. This software process model was used because it felt the most appropriate for our to-do list application, since the system could be divided into small functional components that we can complete one at a time. Not only does it allow the application to be completed in small segments, but it also allows us to manage the project timeline much easier since the incremental development model provides a structured approach for building the application.

e. **List of software requirements:**
   i. **Functional requirements**
      - The system shall allow the user to create a new task by entering a title, description, priority, and due date.
      - The system shall allow the user to edit an existing task.
      - The system shall allow the user to delete a task.
      - The system shall allow the user to mark an existing task as completed.
      - The system shall allow the user to view all tasks.
      - The system shall allow the user to filter tasks by status (e.g., completed or pending).
      - The system shall allow the user to sort tasks by due date.
      - The system shall allow the user to sort tasks by priority.
      - The system shall allow the user to search for tasks using keywords.
      - The system shall allow the user to create a task category.
      - The system shall allow the user to edit a task category.
      - The system shall allow the user to delete a task category.
      - The system shall allow the user to assign tasks to categories.
   ii. **Non-functional requirements**
      - Usability requirement: The system shall allow a new user to understand and perform basic task operations (add, edit, delete, and complete tasks) within 10 minutes of initial use without formal training.
      - Performance requirement: The system shall display the task list within 2 seconds of a user request.
      - Space requirement: The system shall require no more than 100 MB of disk space for application installation and local storage of task data.

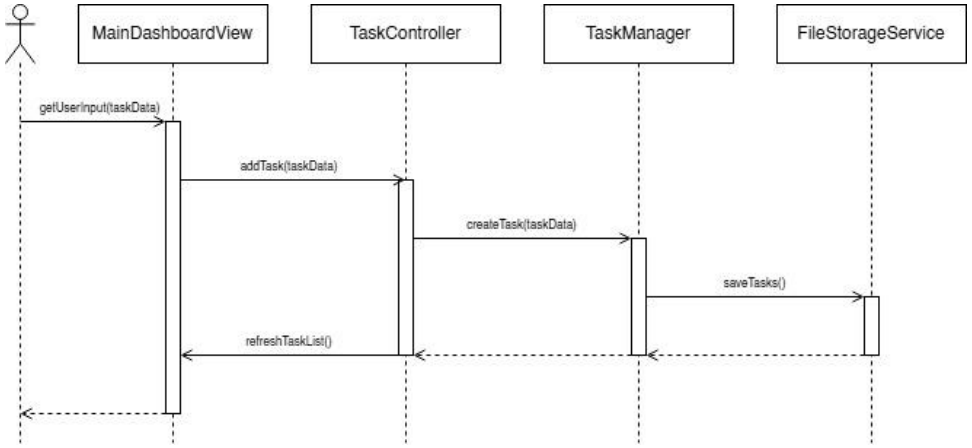
- Efficiency requirement: The system shall allow task operations such as sorting, filtering, and searching tasks to complete within 2 seconds when managing up to 100 stored tasks.
- Dependability requirement: The system shall correctly save all tasks without data loss during normal shutdown.
- Security requirement: The system shall ensure that task data stored locally cannot be modified or accessed by unauthorized applications during normal execution.
- Environmental requirement: The system shall operate on standard desktop operating systems including Windows, macOS, and Linux with Java installed.
- Operational requirement: The system shall support basic mouse and keyboard interactions for all user interface operations.
- Development requirement: The system shall be developed using object-oriented programming principles.
- Regulatory requirement: The system shall follow the software engineering guidelines and requirements specified in the CS 3354 course project specifications.
- Ethical requirement: The system shall not collect or store personally identifiable information beyond user-entered task descriptions.
- Legislative requirement: The system shall store all task data locally and shall not transmit user data to external systems.
- Accounting requirement: The system shall maintain accurate records of task completion status for tracking completed and pending tasks.
- Safety/security requirement: The system shall prevent loss or corruption of task data during unexpected application termination or system crashes.

f. Use case diagram:

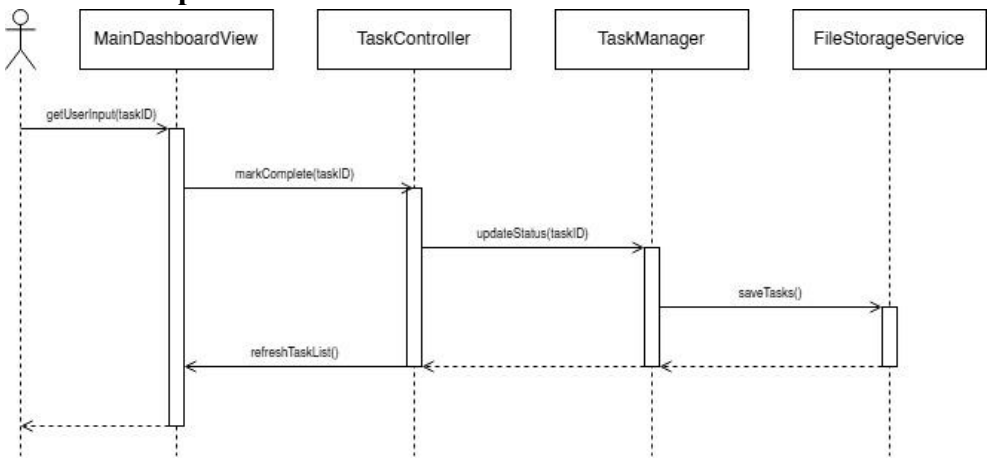


g. Sequence diagrams:

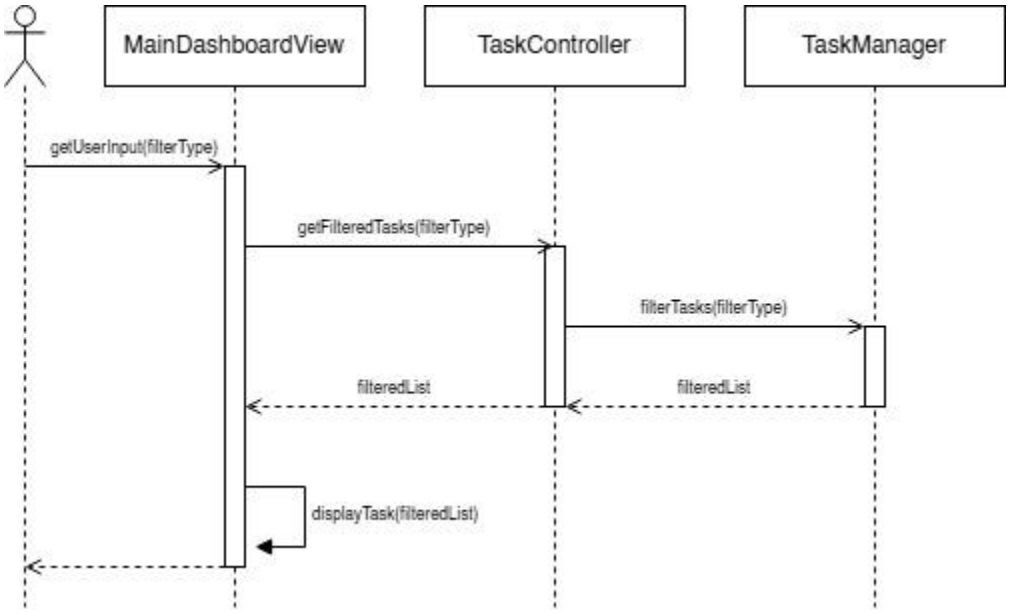
i. Add Task



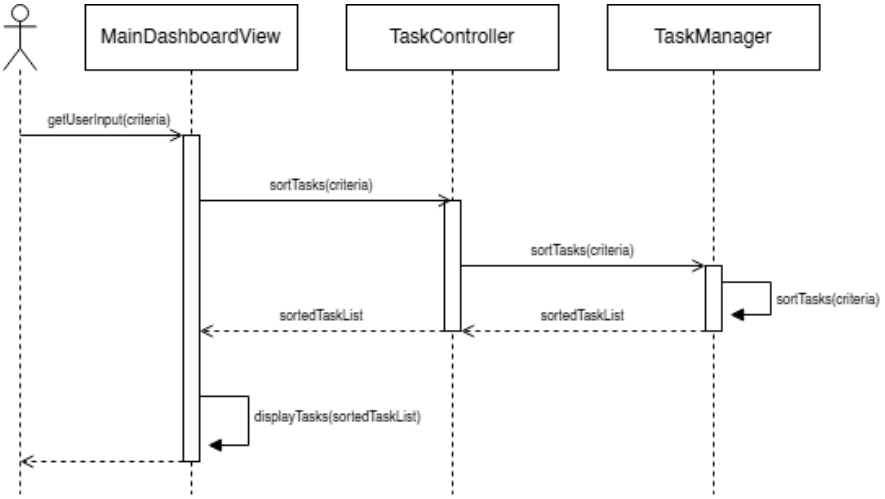
ii. Complete Task



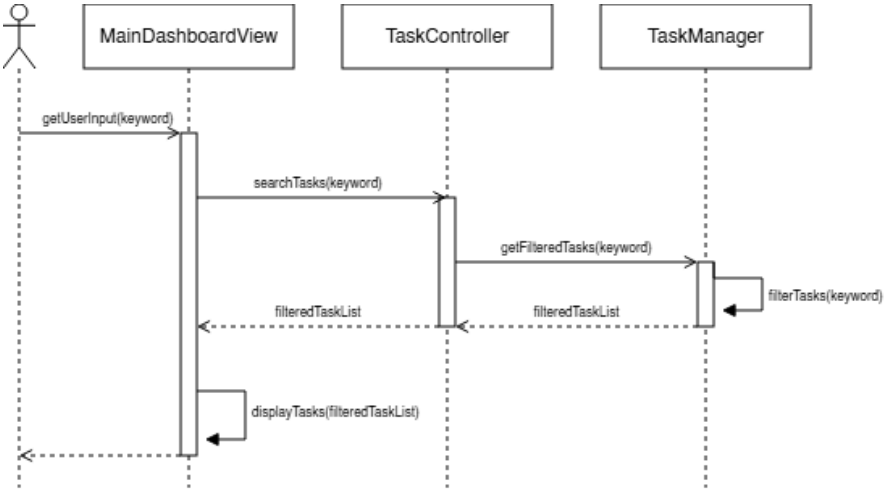
iii. View Tasks



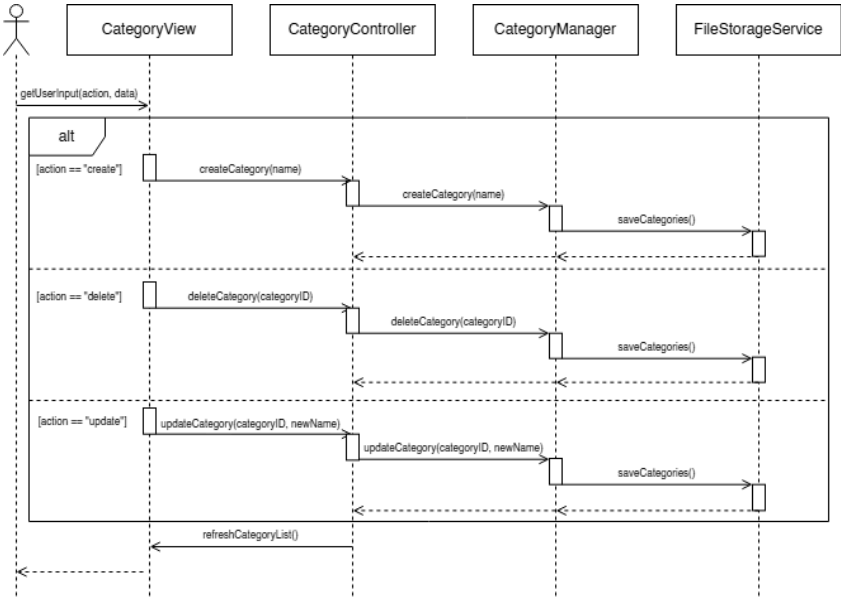
iv. Sort Tasks



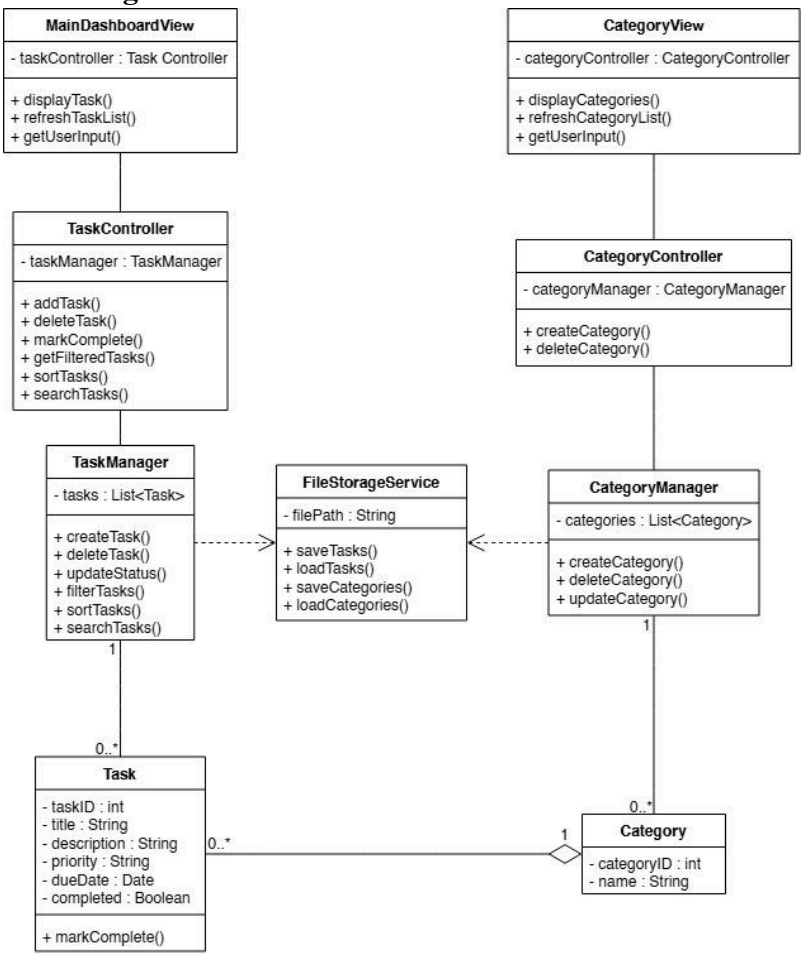
v. Search Tasks



vi. Manage Categories

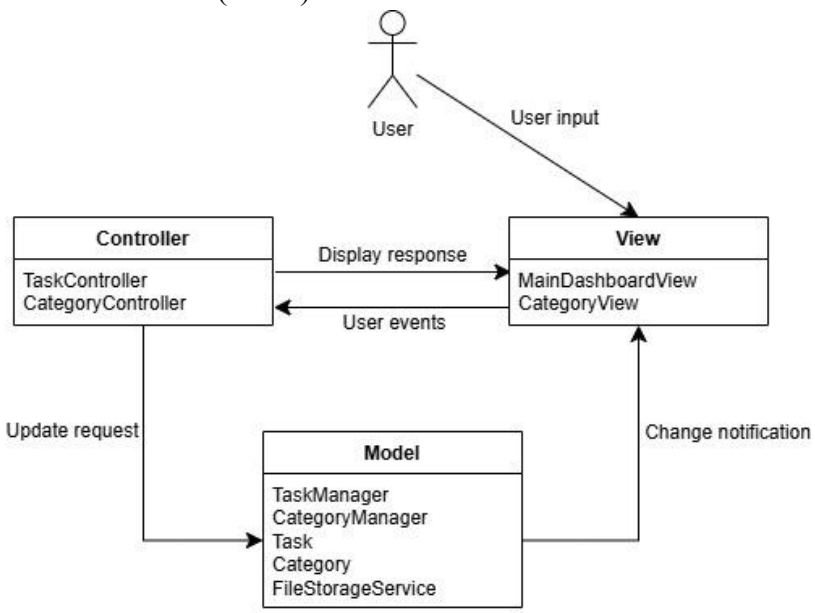


h. Class diagram:



i. Architectural design:

Model-View-Controller (MVC)



4. **Scheduling and Estimation Studies:**
    a. Project Start Date January 6, 2025 (Monday)
    Project End Date April 11, 2025 (Friday) 14 working weeks
    Total Calendar Duration ~96 calendar days (approx. 3 months)
    Weekends Counted? No weekends Saturday and Sunday are excluded from the working schedule
    Working Days per Week 5 days (Monday through Friday)
    Working Hours per Day 6 hours per developer per day
    Total Working Days 70 working days (14 weeks × 5 days)
    Team Size 3-5 developers
    Total Effort (Person-Hours) 1250-2,100 person-hours (5 devs × 70 days × 6 hrs)
    Testing(W1-W2), Design(W3-5), Implementing(W5-W9), Testing(W9-W12), Deploy(W13-W14)

    b. Cost, Effort, and Pricing Estimation
        i. To estimate the cost and pricing of our project, we first determine the system size and development effort, then convert that effort into cost, and finally add a profit margin to determine the price.
        After estimating the total development effort in hours, we multiply it by an average hourly rate to obtain the development cost. Then, we include additional overhead and a profit margin of approximately 20-30% to determine the final project price.
        ii. We used the Function Point (FP) method to estimate system size based on functionality.
        We identified key features of our to-do list application, such as creating, editing, deleting, and viewing tasks, and categorized them into inputs, outputs, and internal data. Based on their low to medium complexity, the system size is estimated at 150-200 Function Points.
        Using an industry average of 10 hours per Function Point, the total effort is 1500-2000 hours, resulting in a development cost of approximately $75,000-$100,000. After adding overhead and a profit margin, the final project price is estimated to be $100,000-$130,000.
    c. Estimated Cost of Hardware Products
        i. The hardware cost for this project is relatively low because the system is designed as a web-based application using cloud infrastructure rather than physical servers.
        We estimate cloud server hosting: $50-$100 per month; database hosting: $20-$50 per month. This results in an annual hardware cost of approximately $1,000-$2,000.
    d. Estimated Cost of Software Products
        i. Most of the software tools used in this project are free or open-source, such as programming languages, frameworks, and version control systems. The only possible costs are optional development tools such as IDE licenses or premium services. We estimate the total software cost to be approximately $500-$1,500.

e. Estimated Cost of Personnel

   i. Personnel cost is the largest component of the project. We estimate a team of 3-5 developers, 1 project manager, and 1 QA tester. Based on the total effort of 1500-2000 hours and average hourly rates, the total personnel cost is approximately $70,000-$90,000, with an additional $2,000-$5,000 for post-deployment training.

5. Test Plan:

This test plan is used to verify the correctness of at least one unit of the To-Do Application. The selected unit for testing is the Task class, which represents an individual task in the application. This unit was selected because task creation and task completion tracking are two core functions of the software.

The test plan focuses on verifying the following behaviors of the Task class:

- a task can be created with a valid name
- an empty task name is rejected
- a task can be marked as completed
- a newly created task defaults to an incomplete state

Testing was performed using automated unit testing in C++ with the GoogleTest framework. Each test case compares the expected behavior of the class against the actual result produced during execution.

A test case is considered successful if the actual result matches the expected result.

```cpp
#include <string>
#include <stdexcept>
#include <gtest/gtest.h>

//fictitious class for a todo app that manages the task entity itself
//made to demo testing capabilities

class Task {
public:
  Task(const std::string& name)
  {
    if (name.empty()) {
      throw std::invalid_argument("Task name cannot be empty");
    }
    else {
      taskName = name;
    }
    isCompleted = false;
  }

  void markCompleted()
  {
    isCompleted = true;
  }

  bool checkTaskCompletion()
  {
    return isCompleted;
  }

  std::string getName()
  {
    return taskName;
  }

private:
  std::string taskName;
  bool isCompleted = false;
};
```

```cpp
//tests using gtest
TEST(TaskTest, ConstructorStoresName)
{
  Task test("Make Dinner");
  EXPECT_EQ(test.getName(), "Make Dinner");
  EXPECT_FALSE(test.checkTaskCompletion());
}

TEST(TaskTest, DoesNotAcceptEmptyName)
{
  EXPECT_THROW(Task(""), std::invalid_argument);
}

TEST(TaskTest, MarkCompleted)
{
  Task test("Make Lunch");
  test.markCompleted();
  EXPECT_TRUE(test.checkTaskCompletion());
}

TEST(TaskTest, TasksDefaultToNotCompleted)
{
  Task test("Make Lunch");
  EXPECT_FALSE(test.checkTaskCompletion());
}
```

```
ryan:~/temp > ./run
Running main() from /usr/src/debug/gtest/googletest-1.17.0/go
[==========] Running 4 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 4 tests from TaskTest
[ RUN      ] TaskTest.ConstructorStoresName
[       OK ] TaskTest.ConstructorStoresName (0 ms)
[ RUN      ] TaskTest.DoesNotExceptEmptyName
[       OK ] TaskTest.DoesNotExceptEmptyName (0 ms)
[ RUN      ] TaskTest.MarkCompleted
[       OK ] TaskTest.MarkCompleted (0 ms)
[ RUN      ] TaskTest.TasksDefaultToNotCompleted
[       OK ] TaskTest.TasksDefaultToNotCompleted (0 ms)
[----------] 4 tests from TaskTest (0 ms total)

[----------] Global test environment tear-down
[==========] 4 tests from 1 test suite ran. (0 ms total)
[  PASSED  ] 4 tests.
```

Test	Description	Expected	Actual	Status
TC-1	Create valid name task	Task is created with valid name and starts as incomplete	Task was created with valid name and starts as incomplete	*PASS*
TC-2	Create empty name task	Task creation should throw "invalid_argument" exception and reject Task	Exception was thrown	*PASS*
TC-3	Mark completed Task	Function checkTaskCompleted returns true once Task has been set as complete	Function returns true as expected	*PASS*
TC-4	Check default completion status upon Task creation	A new Task should default to incomplete	Returns false as expected	*PASS*

- All four automated test cases passed successfully.
- The results show that the selected unit of the software behaves as intended. The Task class correctly accepts valid input, rejects invalid empty input, updates task completion status when requested, and initializes new tasks in an incomplete state.
- This confirms that the tested unit is functioning correctly. This process would then be expanded upon and repeated for all classes and functions within the project

6. **Comparison with Similar Designs:**
   a. The following content includes examples from current to do list applications available for users. Many of them include additional features such as AI integration, time tracking, etc but share the basic and core functionalities of being able to perform basic operational tasks with our application idea. We chose to keep our project's layout simple and minimal for now, however the features from similar examples also serve as good examples to make this project scalable.
      - Zoho Projects:
        - Team based project management
        - Multi user collaboration
        - Time tracking
        - Generation of analytical reports [1]

Both:
- Task creation and management
- Track progress and completion
- Assign deadlines and allow prioritization

- Microsoft To Do:
  - Personal use + small teams
  - Sub tasks
  - File attachments
  - Cross device sync [2]

    Both:
  - Emphasize ease of use
  - Focus on improving productivity
  - Support basic task operations
- Todoist
  - Personal use + small teams
  - Task view options: List, Kanban board, calendar
  - AI integration: paid [3]

    Both:
  - Create, edit, delete tasks
  - Filter and sort tasks
- Google Tasks:
  - Personal use
  - Requires a Google account
  - Integrated with other Google apps like Gmail [4]

    Both:
  - Simple user interface
  - Focus on individual productivity
  - Mark tasks as completed

7. **Conclusion:**
   a. Please make an evaluation of your work.

      Overall, the project was a success. We completed every major phase of the software engineering lifecycle (requirements, design, testing, and a working prototype) and met our goals within the timeline.

      The requirements phase went smoothly. Having clear functional and non-functional requirements early on made the later design work much easier, as the UML diagrams and test planning had a solid foundation to build from. The MVC architecture was a good fit for the project and for the team, since separating the design into Model, View, and Controller allowed members to work on different parts without stepping on each other.

      Testing went well too. All 4 unit tests passed, and the GoogleTest framework made it straightforward to verify the core behavior of the Task class.

      On the whole, the team managed scope well, communicated clearly, and delivered a complete and functional result. The biggest lesson learned was that investing time in strong requirements upfront saved significant time later in the project.

i. Describe any changes that you needed to make (if any), if things have deviated from what you had originally planned.
    1. C++ used for unit testing instead of a Java-based framework
    2. Some features were moved to future work

ii. Give justification for such changes.
    1. The team chose to write unit tests in C++ using GoogleTest rather than using a Java testing framework. This wasn't part of the original plan but made sense in practice since the team was already familiar with GoogleTest, so it was faster and more reliable to use a known tool rather than learn a new one mid-project.
    2. Features like cloud sync, user accounts, and recurring tasks were considered early on but ultimately cut from the current version. With only a limited time, it was the right call to focus on delivering the core features cleanly rather than stretching too thin across too many features.

8. References:

[1]"Time Tracking | Simple Online Time Tracker - Zoho Projects," *Zoho*. https://www.zoho.com/projects/time-tracker.html (accessed Mar. 28, 2026).

[2]"To Do List and Task Management App | Microsoft To Do," www.microsoft.com. https://www.microsoft.com/en-us/microsoft-365/microsoft-to-do-list-app (accessed Mar. 28, 2026).

[3]Todoist, "Todoist | A To-Do List to Organize Your Work & Life," *Todoist.com*, 2024. https://www.todoist.com/ (accessed Mar. 28, 2026).

[4]Google Workspace, "Organize & Track To-Dos with Google Tasks | Google Workspace," *Google Workspace*, 2025. https://workspace.google.com/products/tasks/ (accessed Mar. 28, 2026).